1. Keep unit interfaces small (Parametroak maximo 4-ra limitatu)

1. Hasierako kodea:

```
     public void createReview(Sale sale, String sellerEmail,
String buyerEmail, int stars, String comment) {
         db.getTransaction().begin();
         try {
             Sale s = db.find(Sale.class,
sale.getSaleNumber());
             Registered buyer = db.find(Registered.class,
buyerEmail);
             Registered seller = db.find(Registered.class,
sellerEmail);

             if (s != null && buyer != null && seller != null)
{
                 Review review = new Review(buyer, seller, s,
stars, comment);
                 db.persist(review);
                 buyer.addWrittenReview(review);
                 seller.addReceivedReview(review);
                 s.setReview(review);
                 db.getTransaction().commit();
                 System.out.println(">> Review guardada: De "
+ buyerEmail + " para " + sellerEmail);
             } else {
                 System.err.println("Error: No se pudo crear
la review. Datos incompletos.");
                 db.getTransaction().rollback();
             }
         } catch (Exception e) {
             e.printStackTrace();
             if (db.getTransaction().isActive()) {
                 db.getTransaction().rollback();
             }
         }
     }
```

2. Errefaktorizatutako kodea:

```java
public class ReviewDTO {
    public int stars;
    public String comment;

    public ReviewDTO(int stars, String comment) {
        this.stars = stars;
        this.comment = comment;
    }
}
```

```java
    /**
     * Erosketa baten inguruko balorazio berria sortzen du. Balorazioa erosleari.
     * saltzaileari eta salmentari lotzen zaio.
     * @param sale        Baloratzen den salmenta.
     * @param sellerEmail Saltzailearen posta elektronikoa.
     * @param buyerEmail  Eroslearen (baloratzailearen) posta elektronikoa.
     * @param reviewData TODO
     */
    public void createReview(Sale sale, String sellerEmail, String buyerEmail, ReviewDTO reviewData) {
        db.getTransaction().begin();
        try {
            // 1. Recuperamos los objetos persistentes
            Sale s = db.find(Sale.class, sale.getSaleNumber());
            Registered buyer = db.find(Registered.class, buyerEmail);
            Registered seller = db.find(Registered.class, sellerEmail);

            if (s != null && buyer != null && seller != null) {
                Review review = new Review(buyer, seller, s, reviewData.stars, reviewData.comment);

                db.persist(review);

                buyer.addWrittenReview(review);
                seller.addReceivedReview(review);
                s.setReview(review);

                db.getTransaction().commit();
                System.out.println(">> Review guardada: De " + buyerEmail + " para " + sellerEmail);
            } else {
                System.err.println("Error: No se pudo crear la review. Datos incompletos.");
                db.getTransaction().rollback();
            }
        } catch (Exception e) {
            e.printStackTrace();
            if (db.getTransaction().isActive()) {
                db.getTransaction().rollback();
            }
        }
    }
```

3. Deskribapena: Metodoak parametroen muga gainditzen zuen (5 parametro). Edukia (izarrak eta iruzkina) ReviewDTO Parameter Object baten barruan kapsulatu da, 4 parametro soilik utziz.

4. Egilea: Imanol Egaña

2. Duplicate code (Kode errepikatua ekidin)

1. Hasierako kodea:

```
     public void addToCart(String email, Sale s){
          db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            Sale saleDB = db.find(Sale.class,
s.getSaleNumber());
            if (u != null && saleDB != null) {
                if (!u.isOnCart(saleDB)) {
                    u.addToCart(saleDB);
                }
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
     }

     public void removeFromCart(String email, Sale s){
          db.getTransaction().begin();
        try {
            Registered u = db.find(Registered.class, email);
            Sale saleDB = db.find(Sale.class,
s.getSaleNumber());
            if (u != null && saleDB != null){
               u.getCart().remove(saleDB);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            if (db.getTransaction().isActive())
db.getTransaction().rollback();
        }
     }
```

2. Errefaktorizatutako kodea:

```java
private void updateCartStatus(String email, Sale s, boolean isAdd) {
    db.getTransaction().begin();
    try {
        Registered u = db.find(Registered.class, email);
        Sale saleDB = db.find(Sale.class, s.getSaleNumber());
        if (u != null && saleDB != null) {
            if (isAdd && !u.isOnCart(saleDB)) {
                u.addToCart(saleDB);
            } else if (!isAdd) {
                u.getCart().remove(saleDB);
            }
        }
        db.getTransaction().commit();
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
    }
}

/**
 * Produktu bat erabiltzailearen erosketa-saskira gehitzen du.
 * @param email Erabiltzailearen identifikazio elektronikoa.
 * @param s     Gehitu nahi den salmenta (Sale).
 */
public void addToCart(String email, Sale s){
    updateCartStatus(email, s, true);
}

/**
 * Produktu bat erabiltzailearen erosketa-saskitik kentzen du.
 * @param email Erabiltzailearen identifikazio elektronikoa.
 * @param s     Kendu nahi den salmenta (Sale).
 */
public void removeFromCart(String email, Sale s){
    updateCartStatus(email, s, false);
}
```

3. Deskribapena: Saskira gehitzeko eta kentzeko eragiketa ia berdinak ziren, eta kode egitura bera zeukaten. Errepikatzen zen portaera updateCartStatus metodoan isolatu da.

4. Egilea: Imanol Egaña

3. Write short units of code (Maximo 15 lerro)

1. Hasierako kodea:

```
     public void removeOrder(Order order) {
         db.getTransaction().begin();
         try {
          System.out.println("Buscando orden con título: '" +
order.getTitle() + "'");
             Order o = db.find(Order.class, order.getTitle());

             if (o != null) {
                 Registered reg = o.getRegistered();
                 reg.getOrders().remove(o);
                 o.setRegistered(null);
                 db.remove(o);

                 db.getTransaction().commit();
                 System.out.println("Transacción completada
para: " + o.getTitle());
             } else {
                 db.getTransaction().rollback();
                 System.out.println("No se encontró la orden
en la DB");
             }
         } catch (Exception e) {
             if (db.getTransaction().isActive())
db.getTransaction().rollback();
             e.printStackTrace();
         }
     }
```

2. Errefaktorizatutako kodea:

```java
/**
 * Eskaera bat sistematik ezabatzen du, erabiltzailearekin duen lotura kenduz.
 * @param order Ezabatu nahi den eskaera.
 */
public void removeOrder(Order order) {
    db.getTransaction().begin();
    try {
        System.out.println("Buscando orden con título: '" + order.getTitle() + "'");
        Order o = db.find(Order.class, order.getTitle());

        if (o != null) {
            disconnectOrder(o);

            db.getTransaction().commit();
            System.out.println("Transacción completada para: " + o.getTitle());
        } else {
            db.getTransaction().rollback();
            System.out.println("No se encontró la orden en la DB");
        }
    } catch (Exception e) {
        if (db.getTransaction().isActive()) db.getTransaction().rollback();
        e.printStackTrace();
    }
}

private void disconnectOrder(Order o) {
    Registered reg = o.getRegistered();

    reg.getOrders().remove(o);

    o.setRegistered(null);

    db.remove(o);
}
```

3. Deskribapena: Metodoak lerro gehiegi zituen erlazio bidirekzionalak garbitzeko prozesuaren ondorioz. Beharrezkoak ez ziren inprimatze mezuak kendu dira eta objektuaren deskonexioa disconnectOrder metodora delegatu da, jatorrizkoa 13 lerrotan utziz.

4. Egilea: Imanol Egaña

4. Write simple units of code (Maximo 4 branch points)

1. Hasierako kodea:

```
  public void declineComplaint(Sale s) {
        db.getTransaction().begin();
        try {
            Sale sale = db.find(Sale.class,
s.getSaleNumber());
            if (sale != null && sale.getComplaint() != null) {
                Complaint comp = sale.getComplaint();
                Registered sender = comp.getSender();
                Registered receiver = comp.getReceiver();

                if (sender != null) {

sender.getSentComplaintList().remove(comp);
                }
                if (receiver != null) {

receiver.getReceivedComplaintList().remove(comp);
                }

                sale.setComplaint(null);
                sale.setAlreadyComplaint(false);
                db.remove(comp);
            }
            db.getTransaction().commit();
        } catch (Exception e) {
            db.getTransaction().rollback();
        }
    }
```

2. Errefaktorizatutako kodea:

```java
/**
 * Erreklamazio bat ezesten du. Erreklamazioa ezabatu egiten da
 * eta salmenta egoera berrezartzen da.
 * * @param s Lotutako salmenta.
 */
public void declineComplaint(Sale s) {
    db.getTransaction().begin();
    try { // Erroreak saihesteko
        Sale sale = db.find(Sale.class, s.getSaleNumber());
        if (sale == null || sale.getComplaint() == null) {
            db.getTransaction().commit();
            return; //Guard clause
        }
        Complaint comp = sale.getComplaint();

        if (comp.getSender() != null) {
            comp.getSender().getSentComplaintList().remove(comp);
        }
        if (comp.getReceiver() != null) {
            comp.getReceiver().getReceivedComplaintList().remove(comp);
        }

        sale.setComplaint(null);
        sale.setAlreadyComplaint(false);
        db.remove(comp);

        db.getTransaction().commit();
    } catch (Exception e) {
        db.getTransaction().rollback();
    }
}
```

3. Deskribapena: Logika guztia if (sale != null && sale.getComplaint() != null) baldintzaren barruan habiatu beharrean, egiaztapena alderantzikatu da azkar itzultzeko (Guard Clause erabiliz). Horrela, adarren (branches) sakonera izugarri murriztu da.

4. Egilea: Imanol Egaña